

AgentForge: A Role-Structured Multi-Agent Framework for Repository-Level Software Repair with Execution Feedback

Rajesh Kumar, Waqar Ali, Junaid Ahmed, Najma Imtiaz Ali, Shaban Usman

Revision status. This draft has been rewritten to address the reviewers’ concerns about evaluation leakage, execution semantics, baseline control, and reproducibility. Numerical claims from the rejected version are intentionally not carried forward. Red placeholders must be replaced only with outputs from a leakage-free rerun using the official SWE-bench harness.

Abstract—Repository-level software repair requires more than generating a plausible patch: an agent must inspect a codebase, localize the defect, modify the repository, execute relevant checks, interpret failures, and revise the patch within a limited budget. We present AGENTFORGE, a role-structured multi-agent framework that separates planning, editing, test construction, debugging, and review while maintaining a shared repository state and execution history. The revised design introduces an explicit *post-edit execution invariant*: every state-changing edit must be followed by execution in the same repository environment before another edit is permitted. Agent-generated tests are treated as auxiliary debugging signals rather than ground-truth correctness evidence, and final correctness is measured only by the official SWE-BENCH evaluation harness. We specify a leakage-free protocol that excludes reference patches and benchmark grading metadata from agent-visible inputs, resets episodic memory for each evaluation instance, and compares agent configurations under matched model, tool, token, cost, and execution budgets. **[TBD after leakage-free rerun: Insert the official SWE-bench Lite resolution rate, confidence interval, cost, and controlled-ablation findings.]** The framework and evaluation artifacts will be released with a frozen commit and task-level predictions.

Index Terms—Large language models, multi-agent systems, automated program repair, repository-level software engineering, execution feedback, SWE-bench.

I. INTRODUCTION

Large language models (LLMs) can synthesize code from natural-language specifications, but repository-level software repair remains difficult. A realistic repair task requires the system to interpret an issue report, inspect an unfamiliar repository, identify relevant files, modify existing code, execute tests, diagnose failures, and avoid regressions. Function-level

benchmarks such as HumanEval emphasize isolated synthesis, whereas SWE-BENCH evaluates changes to real repositories under project-specific tests and environments [1, 2].

Tool-using software engineering agents address part of this challenge by exposing shell, editor, search, and execution interfaces to an LLM [3–5]. Multi-agent systems add role specialization, assigning planning, coding, testing, or review to separate agents [6–9]. These directions are complementary: tool access supplies interaction with the repository, while decomposition can make intermediate responsibilities explicit. However, claims about the value of execution or multi-agent structure require carefully controlled evaluation. A system with more calls, more tokens, more tools, and more test attempts cannot be fairly compared with a weaker single-call baseline without controlling those resources.

This paper studies a narrower question: *under matched computational and tool budgets, does a role-structured workflow with enforced post-edit execution improve repository-level repair?* We introduce AGENTFORGE, a framework with five roles: Planner, Coder, Tester, Debugger, and Critic. The central design feature is not merely that execution tools are available, but that the orchestrator treats execution as a state-machine constraint. After a Coder or Debugger action changes the repository, the system must execute a validation command and record its outcome before another state-changing action can occur.

The revised evaluation is designed around four validity requirements. First, the agent receives only the issue statement and the base repository checkout; reference patches, reference-derived file paths, benchmark test metadata, and grading labels are excluded. Second, execution occurs in the same instance-specific repository environment used during interaction, while final scoring is performed independently through the official SWE-BENCH harness. Third, generated tests are auxiliary hypotheses and are not equated with benchmark correctness. Fourth, comparisons are paired and budget controlled.

Contributions. This work makes the following contributions:

- A role-structured repository-repair architecture that separates planning, editing, auxiliary test generation, debugging, and review while preserving a single mutable repository state.
- An explicit post-edit execution invariant enforced by the orchestrator rather than delegated to a planner-generated sequence.
- A contamination-resistant evaluation protocol that excludes gold-patch information, isolates episodic memory,

R. Kumar is with the International Research Center for Complexity Sciences, Hangzhou International Innovation Institute, Beihang University, Hangzhou 311115, China (e-mail: rajakumarlohano@gmail.com).

W. Ali is with the Department of Computer Science, College of Science, Mathematics and Technology, Wenzhou-Kean University, Wenzhou 325060, China (e-mail: waqar.uestc@yahoo.com).

J. Ahmed is with the Department of Computer Systems Engineering, Sukkur IBA University, Sindh, Pakistan (e-mail: j.bhatti@iba-suk.edu.pk).

N. I. Ali is with the Faculty of Information and Communication Technology, Universiti Teknikal Malaysia Melaka, Melaka 76100, Malaysia (e-mail: najma@utem.edu.my).

S. Usman is with the Yibin Park of the University of Electronic Science and Technology of China, Yibin 644000, China (e-mail: shabanusman@yahoo.com).

and uses the official SWE-BENCH evaluation infrastructure.

- A controlled experimental design that separates the effects of role decomposition, forced execution, tool access, and additional test-time computation under matched budgets.
- **[TBD after leakage-free rerun: Summarize only findings supported by the new task-level rerun.]**

The paper deliberately avoids claiming that execution-enabled agents are new. SWE-agent and OpenHands already support iterative interaction with executable environments [3, 4]. Our contribution is the orchestration policy, its controlled evaluation, and the separation of auxiliary runtime signals from hidden benchmark correctness.

II. RELATED WORK

A. Code Generation and Automated Program Repair

Large pretrained models such as Codex, AlphaCode, StarCoder, and Code Llama established strong code-generation capabilities [1, 10–12]. Automated program repair (APR), however, requires a candidate patch to satisfy behavioral constraints in an existing system. Classical APR systems search or synthesize patches using tests and fault-localization signals [13]. More recent LLM-based repair systems use error messages, execution traces, or iterative self-debugging to refine outputs [14–18]. These methods motivate the use of runtime evidence, but generated or visible tests can provide incomplete supervision and must not be conflated with hidden benchmark correctness.

B. Tool-Using Software Engineering Agents

ReAct interleaves reasoning and tool use, and Reflexion adds memory-based self-correction [19, 20]. SWE-agent provides an agent-computer interface for repository navigation, editing, and command execution [3]. OpenHands offers an open platform with sandboxed runtime interaction [4], while AutoCodeRover combines repository analysis with patch generation [5]. These systems demonstrate that executable environments are already central to autonomous software engineering. Accordingly, AGENTFORGE does not claim novelty for sandbox access itself; it studies an orchestrator-enforced execution schedule under controlled budgets.

C. Multi-Agent Software Engineering

MetaGPT and ChatDev model software development as communication among specialized roles [8, 9]. AutoGen provides a general framework for multi-agent interaction [21]. MAGIS applies manager, repository-custodian, developer, and QA roles to GitHub issue resolution [6]; AgentMesh similarly explores cooperative role decomposition [7]. Role specialization can improve modularity and interpretability, but additional agents also increase test-time computation. A convincing evaluation must therefore compare role structures while holding the base model, tool set, context, and resource budget constant.

D. Repository Retrieval and Memory

Repository-level generation benefits from retrieving relevant files and functions [22, 23]. Retrieval-augmented generation more broadly conditions model outputs on external evidence [24]. Episodic memory may transfer useful patterns across tasks, as in Reflexion [20], but evaluation-time memory creates contamination risks when benchmark instances influence later instances. In our primary evaluation, memory is empty and reset for every task. Cross-task memory is treated as a separate experiment and may be populated only from a documented, disjoint training set.

III. AGENTFORGE

A. Problem Definition and Information Boundary

For each task i , the agent receives an issue description q_i and a repository checkout R_i^0 at the benchmark base commit. The agent produces a patch p_i that transforms the repository into $R_i^* = R_i^0 \oplus p_i$. The final patch is successful only if the official benchmark harness accepts it.

The following fields are prohibited from all agent-visible inputs and retrieval indexes during inference:

- the reference or gold patch;
- file paths extracted from the reference patch;
- benchmark grading labels or expected outcomes;
- FAIL_TO_PASS and PASS_TO_PASS identifiers;
- data derived from other evaluation instances.

The agent may inspect only the base repository, the issue statement, command outputs produced during its own run, and optional memory built from a disjoint training set.

B. Roles and Shared State

AGENTFORGE contains five role-specific agents coordinated by a deterministic Orchestrator:

a) Planner.: The Planner converts the issue description and retrieved repository context into a structured repair plan. The plan may identify investigation steps and candidate validation commands, but it cannot waive the execution invariant.

b) Coder.: The Coder proposes a minimal patch against the current repository state. Existing files are edited through unified diffs where possible.

c) Tester.: The Tester may construct auxiliary tests or reproduction scripts from the issue description and inspected code. These artifacts are used for diagnosis only and are labeled as model-generated evidence.

d) Debugger.: The Debugger receives the current patch, command, exit status, standard output, and standard error. It proposes a revised patch targeted at the observed failure.

e) Critic.: The Critic reviews the issue, cumulative diff, executed commands, and recorded outcomes. It may stop the search or request another repair iteration, but it cannot declare benchmark success. Only the official harness determines resolution.

All roles operate on one mutable repository state and one append-only execution history. This avoids parallel agents silently reasoning over inconsistent versions of the codebase.

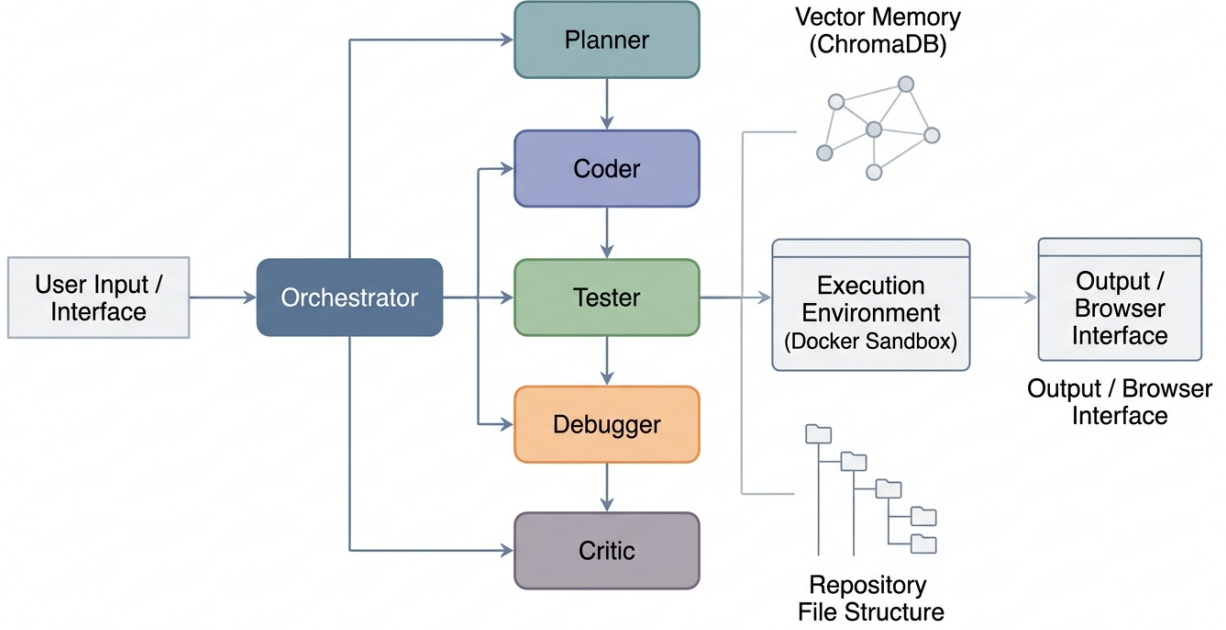


Fig. 1. Conceptual architecture of AGENTFORGE. For benchmark evaluation, the repository is initialized from the base commit, cross-instance memory is disabled, and all state-changing edits are followed by execution in the same task environment. The figure is architectural; the official benchmark harness independently evaluates the final patch.

C. Post-Edit Execution Invariant

Let R_t denote the repository state at step t . A state-changing action a_t applies a patch and produces R_{t+1} . The Orchestrator maintains a Boolean flag x_t indicating whether execution is required. The invariant is

$$R_{t+1} \neq R_t \Rightarrow x_{t+1} = 1, \quad (1)$$

and no subsequent state-changing action is permitted while $x = 1$. The flag is cleared only after a command is executed in the same repository environment and its outcome is appended to the history. Thus, execution is enforced by control flow rather than by hoping that the Planner schedules a Tester step.

D. Execution Signals and Test Separation

The system distinguishes three forms of evidence:

- 1) **Repository commands**, such as targeted project tests, linters, static checks, or reproduction scripts available from the base repository.
- 2) **Agent-generated tests**, which are diagnostic hypotheses and may be wrong, incomplete, or overfitted to the candidate patch.
- 3) **Official benchmark tests**, which are hidden from the agent and used only by the independent evaluation harness.

A successful auxiliary command indicates that a candidate survived a particular check; it does not establish ground-truth correctness. The manuscript therefore uses the term *execution feedback*, not *ground-truth feedback*, for agent-visible runtime outcomes.

Algorithm 1 AgentForge Repair Loop with Enforced Post-Edit Execution

Require: Issue q , base repository R^0 , budget B
Ensure: Final patch p and complete trajectory H

- 1: $R \leftarrow R^0$; $H \leftarrow []$; memory $\leftarrow \emptyset$
- 2: $C \leftarrow \text{RETRIEVEBASEREPOCONTEXT}(q, R)$
- 3: $P \leftarrow A_{\text{plan}}(q, C)$
- 4: $p \leftarrow \emptyset$; $b \leftarrow 0$
- 5: **while** $b < B$ **do**
- 6: $a \leftarrow \text{CHOOSEEDITROLE}(P, H)$
- 7: $\Delta \leftarrow A_a(q, R, H)$ $\{a \in \{\text{coder, debugger}\}\}$
- 8: $R \leftarrow \text{APPLYPATCH}(R, \Delta)$; $p \leftarrow \text{DIFF}(R^0, R)$
- 9: $x \leftarrow 1$
- 10: $c \leftarrow \text{SELECTVALIDATIONCOMMAND}(q, R, H)$
- 11: $(z, o, e) \leftarrow \text{EXECUTE}(R, c)$
- 12: $H.\text{APPEND}(\Delta, c, z, o, e)$; $x \leftarrow 0$; $b \leftarrow b + 1$
- 13: **if** $z = 0$ **then**
- 14: $v \leftarrow A_{\text{critic}}(q, p, H)$
- 15: **if** $v = \text{STOP}$ **then**
- 16: **return** (p, H)
- 17: **end if**
- 18: **end if**
- 19: **end while**
- 20: **return** (p, H)

To assess generated-test quality separately, generated tests should be executed on both the base repository and a known-correct patched repository drawn from a disjoint development set. Reported metrics should include the proportion that fail

on the buggy version, pass on the fixed version, and avoid unrelated failures. **[TBD after leakage-free rerun: Insert the pre-registered generated-test validation protocol and results.]**

E. Repository Retrieval and Episodic Memory

The repository index is constructed only from files in the base checkout. Retrieval queries use the issue statement and inspected execution history; no query includes reference-patch content. The index configuration, file extensions, chunk size, overlap, embedding model, maximum retrieved tokens, and truncation order must be reported in full.

The primary benchmark condition uses no cross-instance episodic memory. The memory store is empty at the beginning of every task and discarded afterwards. A secondary memory experiment may use a frozen store constructed only from a disjoint training or development split. Evaluation trajectories and Critic judgments are never written back into memory during benchmark inference.

F. Sandbox and Repository Environment

Interactive commands must run inside the benchmark instance’s repository-specific environment, including its checked-out base commit and declared dependencies. A generic Python image alone is insufficient for many SWE-BENCH tasks. The final implementation should therefore provision the same instance image family used by the official harness or a documented compatible environment for interaction, while final scoring remains delegated to the official harness.

Docker reduces accidental host interference but is not, by itself, a complete security boundary. The implementation should report the container user, mounted filesystems, Linux capabilities, seccomp or AppArmor profile, PID and memory limits, network policy, timeouts, and cleanup procedure. We make no claim of production-grade isolation solely from using Docker.

IV. EXPERIMENTAL DESIGN

A. Benchmark

We evaluate on SWE-BENCH Lite, a 300-instance subset of real GitHub issues and repository snapshots [2]. Each task is considered resolved only when the official harness applies the generated patch and all required regression and repair checks succeed. Patch-application rate is reported separately and, by definition, cannot be lower than resolution rate.

B. Leakage-Free Evaluation Protocol

Before inference, the evaluation launcher creates the task environment from the base commit and passes only the issue statement and repository access to the agent. Reference patches and benchmark test metadata remain outside the agent process. The final patch is serialized in the official prediction format and evaluated through the official SWE-BENCH harness. The agent process cannot inspect evaluator outputs for the same task and cannot modify its patch after official scoring.

The release will include:

- a frozen repository commit and container image identifiers;
- the exact benchmark version and instance list;
- task-level prediction JSONL files;
- task-level trajectories, token counts, execution counts, wall-clock time, and API cost;
- scripts that reproduce every table and confidence interval.

C. Controlled Comparisons

The primary factorial comparison separates role decomposition from execution scheduling. All configurations use the same base model, issue statement, repository access, tool set, context-window limit, token budget, API-cost ceiling, wall-clock limit, and maximum number of environment executions.

A no-tool, single-call language-model baseline may be reported as a lower bound, but it is not used to isolate the effect of multi-agent structure or execution. Published results from SWE-agent, OpenHands, MAGIS, or other systems are presented in a separate contextual table because their models, budgets, prompts, and environments differ. We do not apply paired significance tests to aggregate results taken from other publications.

D. Budget and Implementation Controls

For every locally rerun configuration, report:

- exact model name and API version;
- sampling parameters and retry policy;
- maximum input and output tokens;
- maximum API cost per task;
- maximum wall-clock time and execution attempts;
- repository commands available to the agent;
- number and size of retrieved context chunks;
- whether auxiliary tests are enabled.

When an API does not guarantee deterministic sampling, repeated runs should be treated as repeated stochastic trials rather than claimed as exact replication from a seed alone.

E. Metrics and Statistical Analysis

The primary metric is official task resolution. Secondary metrics include patch-application rate, token usage, API cost, wall-clock time, number of model calls, number of environment executions, and cost per resolved task. We report paired task-level differences for locally rerun configurations.

For a method evaluated on n tasks, uncertainty in resolution rate is reported with a bootstrap or exact binomial confidence interval. McNemar’s test is used only when per-instance binary outcomes are available for both methods on the same task set. Cost comparisons report mean, median, and interquartile range because agent trajectories are typically skewed.

V. RESULTS

This section is intentionally structured but not populated with results from the rejected evaluation. All values below must be generated from the corrected implementation and official harness.

TABLE I

PRIMARY CONTROLLED CONFIGURATIONS. “FORCED EXECUTION” MEANS THAT THE ORCHESTRATOR BLOCKS FURTHER EDITS UNTIL A POST-EDIT COMMAND HAS RUN AND ITS OUTCOME HAS BEEN RECORDED.

Configuration	Role structure	Repository tools	Iterative edits	Forced execution	Matched budget
Single agent, optional execution	Single	Yes	Yes	No	Yes
Single agent, forced execution	Single	Yes	Yes	Yes	Yes
Multi-agent, optional execution	Five roles	Yes	Yes	No	Yes
AGENTFORGE	Five roles	Yes	Yes	Yes	Yes

TABLE II

OFFICIAL SWE-BENCH LITE RESULTS UNDER MATCHED BUDGETS. CONFIDENCE INTERVALS AND COST STATISTICS MUST BE COMPUTED FROM RELEASED TASK-LEVEL RECORDS.

Method	Resolved	Resolve rate	Patch apply rate	Mean cost
Single, optional execution	[TBD after leakage-free rerun: α]	[TBD after leakage-free rerun: rate (CI)]	[TBD after leakage-free rerun: rate]	[TBD after leakage-free rerun: USD]
Single, forced execution	[TBD after leakage-free rerun: α]	[TBD after leakage-free rerun: rate (CI)]	[TBD after leakage-free rerun: rate]	[TBD after leakage-free rerun: USD]
Multi-agent, optional execution	[TBD after leakage-free rerun: α]	[TBD after leakage-free rerun: rate (CI)]	[TBD after leakage-free rerun: rate]	[TBD after leakage-free rerun: USD]
AGENTFORGE	[TBD after leakage-free rerun: α]	[TBD after leakage-free rerun: rate (CI)]	[TBD after leakage-free rerun: rate]	[TBD after leakage-free rerun: USD]

TABLE III
RESOURCE USE PER TASK.

Method	Tokens	Calls	Executions	Median cost (IQR)
Single, optional execution	[TBD after leakage-free rerun:]	[TBD after leakage-free rerun:]	[TBD after leakage-free rerun:]	[TBD after leakage-free rerun:]
Single, forced execution	[TBD after leakage-free rerun:]	[TBD after leakage-free rerun:]	[TBD after leakage-free rerun:]	[TBD after leakage-free rerun:]
Multi-agent, optional execution	[TBD after leakage-free rerun:]	[TBD after leakage-free rerun:]	[TBD after leakage-free rerun:]	[TBD after leakage-free rerun:]
AGENTFORGE	[TBD after leakage-free rerun:]	[TBD after leakage-free rerun:]	[TBD after leakage-free rerun:]	[TBD after leakage-free rerun:]

A. Main Results

The primary analysis should answer two paired questions. First, does forced post-edit execution improve performance when the role structure is held constant? Second, does role decomposition improve performance when execution scheduling is held constant? Any interpretation must account for confidence intervals, paired outcomes, and resource use rather than percentage differences alone.

B. Equal-Budget and Cost-Effectiveness Analysis

A method that uses more computation may still be useful, but the manuscript should describe the tradeoff directly. Report cost per resolved task and resolution at fixed cost ceilings rather than characterizing a more expensive system as more sample efficient without evidence.

C. Ablation Study

Agent-removal ablations must specify exactly how the pipeline changes. Removing a role should not silently alter the number of calls, tool permissions, execution opportunities, or context budget. A preferred design reallocates the removed role’s budget to the remaining configuration or reports both fixed-architecture and equal-budget ablations.

D. Failure Analysis

Failure analysis should be conducted on a pre-specified random or stratified sample of unresolved tasks by at least two annotators using a documented codebook. Suggested categories include localization error, incomplete semantic repair, invalid patch, dependency or environment failure, overfitting to auxiliary tests, repeated low-diversity repair, and budget exhaustion. Report inter-rater agreement and allow multi-label coding where causes overlap. **[TBD after leakage-free rerun: Insert adjudicated counts, examples, and agreement statistics.]**

TABLE IV

PRE-SPECIFIED ABLATIONS UNDER AN EQUAL RESOURCE BUDGET.

Configuration	Resolve rate	Paired difference
Full AGENTFORGE	TBD	—
Without Planner	TBD	TBD
Without Tester	TBD	TBD
Without Debugger	TBD	TBD
Without Critic	TBD	TBD
Without repository retrieval	TBD	TBD
Without generated tests	TBD	TBD

VI. THREATS TO VALIDITY

a) *Construct validity.*: Official SWE-BENCH resolution is a binary measure of benchmark success, not a complete measure of software quality, maintainability, security, or developer productivity. Agent-generated tests provide partial diagnostic evidence and may be correlated with the candidate patch.

b) *Internal validity.*: Agent systems are sensitive to prompts, model versions, context truncation, execution budgets, and transient API behavior. The revised study controls these factors within local comparisons and releases task-level trajectories. It does not infer causal effects from comparisons with published aggregate scores produced under different conditions.

c) *External validity.*: SWE-BENCH Lite consists primarily of Python repositories and may not represent other languages, build systems, or industrial workflows. A single benchmark also cannot establish general superiority across software engineering tasks.

d) *Memory and contamination.*: Cross-task memory can leak evaluation information even without explicit reference patches. The primary study therefore resets memory per instance. Any memory-enabled experiment must use a frozen, documented, disjoint source corpus.

e) *Security.*: Resource-limited containers reduce but do not eliminate risks from untrusted generated code. The system is an experimental research artifact and should not be treated as a production security boundary.

VII. CONCLUSION

We presented a revised formulation of AGENTFORGE as a role-structured framework for repository-level software repair. The system separates planning, editing, auxiliary test generation, debugging, and review, while an orchestrator-enforced invariant requires execution after every repository-changing edit. The paper deliberately distinguishes runtime evidence available to the agent from hidden benchmark correctness and specifies a leakage-free, budget-controlled evaluation using

the official SWE-BENCH harness. **[TBD after leakage-free rerun: Insert only conclusions supported by the corrected task-level evaluation.]** The broader lesson is methodological: claims about agent architecture require transparent information boundaries, identical evaluation infrastructure, paired outcomes, and explicit accounting for test-time computation.

APPENDIX A AGENT PROMPTS

The following prompts reflect the role contracts in the evaluated implementation. The camera-ready artifact should include these prompts as plain-text files in a frozen repository commit and record any task-specific wrappers or formatting performed by the evaluation launcher.

A. Planner

You are a senior software architect. Given a task, produce a JSON object with two keys: “explanation,” a one-sentence description of the approach, and “steps,” an array of objects. Each step specifies an agent, description, and optional file. Return only valid JSON, with no markdown fences or additional text.

B. Coder: New Implementation

You are an expert software engineer. Write clean, well-commented, production-ready Python code.

C. Coder: Existing File

You are an expert at producing minimal unified diffs in git-diff format. Return only the diff, with no explanation.

D. Tester

You are an expert in pytest and test-driven development. Generate comprehensive pytest test cases, including edge cases and exceptions. Return only valid Python code, with no explanations or markdown fences.

E. Debugger

You are an expert debugger. Given code and an error message, return a corrected version of the code. Return only the corrected code, with no explanation.

F. Critic

You are a senior code reviewer. Evaluate the results of an AI coding pipeline. Return “PASS” if the output appears correct and complete, or “FAIL: <reason>” otherwise. In the benchmark implementation, this judgment controls search termination but is not treated as official correctness.

APPENDIX B REPRODUCIBILITY CHECKLIST

Before submission, the artifact should provide all of the following:

- frozen source commit and dependency lock files;
- exact prompt files and prompt-construction code;
- benchmark version, split, and instance IDs;
- proof that reference patches and grading metadata are inaccessible to the agent;

- instance-specific environment identifiers;
- official harness command and prediction JSONL;
- raw per-task outcomes for every local baseline;
- token, call, execution, latency, and cost logs;
- scripts for tables, figures, confidence intervals, and paired tests;
- documented memory initialization and contamination controls.

REFERENCES

- [1] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, “Evaluating large language models trained on code,” 2021, codex / GitHub Copilot paper.
- [2] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan, “SWE-bench: Can language models resolve real-world github issues?” in *The Twelfth International Conference on Learning Representations*, 2024, ICLR 2024 Oral.
- [3] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “Swe-agent: Agent-computer interfaces enable automated software engineering,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024, neurIPS 2024.
- [4] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh *et al.*, “Openhands: An open platform for ai software developers as generalist agents,” 2024, primary open-source baseline for multi-agent SWE (also accepted as ICLR 2025 poster).
- [5] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, “Autocoderover: Autonomous program improvement,” 2024, ISSTA 2024.
- [6] W. Tao *et al.*, “Magis: Llm-based multi-agent framework for github issue resolution,” 2024.
- [7] S. Khanzadeh, “Agentmesh: A cooperative multi-agent generative ai framework for software development automation,” 2025.
- [8] S. Hong *et al.*, “Metagpt: Meta programming for a multi-agent collaborative framework,” 2023.
- [9] C. Qian *et al.*, “Chatdev: Communicative agents for software development,” 2023.
- [10] Y. Li *et al.*, “Competition-level code generation with alphacode,” 2022.
- [11] R. Li *et al.*, “Starcode: May the source be with you!” 2023.
- [12] B. Rozière *et al.*, “Code llama: Open foundation models for code,” 2023.
- [13] C. Le Goues, T. Nguyen, S. Forrest, and W. Weimer, “Genprog: A generic method for automatic software repair,” *IEEE Transactions on Software Engineering (TSE)*, vol. 38, no. 1, pp. 54–72, 2012.
- [14] C. Xia and L. Zhang, “Less is more: Summary of long code for repair,” in *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2022.
- [15] M. Jin *et al.*, “Inferfix: End-to-end program repair with llms,” in *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, 2023.

- [16] Z. Fan *et al.*, “Automated program repair in the era of large pre-trained language models,” 2023.
- [17] X. Chen *et al.*, “Teaching large language models to self-debug,” 2023.
- [18] T. X. Olausson *et al.*, “Is self-repair a silver bullet for code generation?” 2023, justification for the Iterative Debug Loop in AgentForge.
- [19] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [20] N. Shinn *et al.*, “Reflexion: Language agents with verbal reinforcement learning,” 2023.
- [21] Q. Wu *et al.*, “Autogen: Enabling next-gen llm applications via multi-agent conversation,” 2023.
- [22] F. Zhang *et al.*, “Repocoder: Repository-level code completion,” 2023.
- [23] D. Shrivastava *et al.*, “Repofusion: Training code models on whole repositories,” 2023.
- [24] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” 2020.